rump

A Reference Manual for the Multiprecision Arithmetic Crate

version 0.2.2

August 2026

Abstract

rump is a dependency-free Rust crate of multiprecision integer arithmetic implemented from the literature: Knuth's Algorithm D for division, Montgomery and Barrett modular reduction, Karatsuba and Toom–Cook multiplication, Lehmer's gcd with subquadratic Half-GCD, the Jacobi symbol by quadratic reciprocity, Tonelli–Shanks and Cipolla square roots, Miller–Rabin and Baillie–PSW primality, binary fields $\mathbb{F}_{2^m}$, univariate polynomials over $\mathbb{Z}$ and $\mathbb{F}_p$ with factorization, and integral LLL lattice reduction. This manual documents every public primitive: the mathematics that defines it, the algorithm that computes it, and a worked example of its use. Code examples are pinned by the crate's test suite.

Contents

1 Preliminaries **2**
- 1.1 Scope and intended use . . . 2
- 1.2 Imports and naming . . . 3
- 1.3 Notation . . . 3

2 Unsigned integers: `BigUint` **3**
- 2.1 Construction, bytes, and radix strings . . . 3
- 2.2 Comparison and predicates . . . 4
- 2.3 Addition and subtraction . . . 4
- 2.4 Multiplication . . . 4
- 2.5 Division and remainder . . . 5
- 2.6 Shifts and bit access . . . 6
- 2.7 Integer roots . . . 6
- 2.8 Size estimates . . . 6

3 Signed integers: `BigInt` and `Sign` **7**

4 Barrett reduction: `BarrettCtx` **7**

5 The Montgomery domain: `MontgomeryCtx` **8**

6 Binary fields: `Gf2m` **10**
6.1 Arithmetic . . . 10
6.2 Trace, half-trace, square roots, and quadratics . . . 10
6.3 Irreducibility: Rabin's test . . . 11

7 Number theory **12**
7.1 Greatest common divisors . . . 12
7.2 Quadratic-residue symbols . . . 12
7.3 Modular exponentiation and inverses . . . 13
7.4 Square roots modulo a prime . . . 14
7.5 Square roots modulo a prime power . . . 15
7.6 The Chinese Remainder Theorem . . . 15
7.7 Valuation . . . 15
7.8 Rational reconstruction . . . 16
7.9 Product trees, remainder trees, and batch smoothness . . . 16
7.10 Primality . . . 17

8 Polynomials: `PolyZ` and `PolyModP` **18**
8.1 Arithmetic over $\mathbb{Z}$ . . . 19
8.2 Division over $\mathbb{Z}$ . . . 19
8.3 Resultants and discriminants . . . 19
8.4 Arithmetic over $\mathbb{F}_p$ . . . 20
8.5 Factorization over $\mathbb{F}_p$ . . . 20

9 Lattice reduction: `lll_reduce` **21**

10 Random sampling **22**

11 Panics **24**

1 Preliminaries

1.1 Scope and intended use

Two properties define the intended use of every primitive in this manual.

Contract 1.1 (Variable time)**.** *Operations take data-dependent paths and their running time may leak their operands. Do not use rump where timing must not leak secrets.*

Contract 1.2 (Non-secret data)**.** *rump is not a secret-scrubbing or constant-time library. As inexpensive defense in depth, every `BigUint` volatile-wipes its live limbs on drop and the Montgomery exponentiation ladder wipes its workspaces on exit; that is the extent of it. Spare capacity and buffers freed on reallocation are not wiped, and `Debug` prints every limb. A consumer handling key material must add cryptographic memory hygiene at its own layer.*

The crate is supported on 64-bit targets only: the limb layout and the index arithmetic that sizes it assume a 64-bit `usize`.

1.2 Imports and naming

The crates.io package is `rust-mp`; the library target is `rump`, so code always says `use rump::...`:

```
[dependencies]
rust-mp = "0.2"
```

```
use rump::{
    crt_combine, gcd, gcd_extended, is_probable_prime, is_probable_prime_bpsw,
    is_probable_prime_with_bases, is_strong_lucas_probable_prime, jacobi,
    kronecker, lcm, legendre, miller_rabin_witness, mod_inverse,
    mod_inverse_batch, mod_pow, primes_below, product_tree, random_below,
    random_coprime_below, random_nonzero_below, random_probable_prime,
    rational_reconstruct, rational_reconstruct_bounded, remainder_tree,
    remove_factor, smooth_parts, sqrt_mod, sqrt_mod_prime_power, valuation,
    lll_reduce, lll_reduce_delta,
    BarrettCtx, BigInt, BigUint, Gf2m, MontgomeryCtx, PolyModP, PolyZ, Rng, Sign,
};
```

This manual documents the development tree that ships as version 0.2.2. Two behaviours described here are new at that version and absent from earlier releases: the samplers' stall guards (§10) and the unbalanced multiplication kernel (§2.4).

1.3 Notation

Throughout, $\beta = 2^{64}$ is the limb base. A non-negative integer a is represented as a little-endian limb vector

$$a \;=\; \sum_{i=0}^{k-1} a_i\, \beta^{\,i}, \qquad 0 \le a_i < \beta, \tag{1}$$

normalized so the most significant limb is non-zero (zero is the empty vector). $\lfloor x \rfloor$ is the floor, $\lg x = \log_2 x$, and $\left(\frac{a}{n}\right)$ is the Jacobi (or, over a prime, Legendre) symbol. Polynomial rings are written $\mathbb{Z}[x]$ and $\mathbb{F}_p[x]$; $\operatorname{lc}(f)$ is the leading coefficient of f and $\deg f$ its degree, with $\deg 0 = -\infty$.

2 Unsigned integers: `BigUint`

2.1 Construction, bytes, and radix strings

`zero`, `one`, `from_u64`, and `from_u128` build small values. `from_be_bytes` / `to_be_bytes` are the one external binary format — big-endian bytes, as DER, PEM, and the RFC wire formats use; `to_be_bytes_padded(w)` zero-pads on the left to a fixed width and panics when the value does not fit. `low_u128` and `low_bits(k)` $(= a \bmod 2^k)$ read the low end directly.

`from_str_radix` / `to_str_radix` convert against any radix $2 \le r \le 36$; `Display` and `FromStr` are the base-10 special case. Power-of-two radices convert by direct bit packing. The rest run classical word-sized conversion at small sizes and switch to divide-and-conquer against a ladder of squared radix powers above measured crossovers: to parse n digits, split at the largest r^{2^j} below the value, giving the recurrence $T(n) = 2\,T(n/2) + M(n)$, where M is the multiplication cost — subquadratic wherever M is.

```
let n = BigUint::from_str_radix("deadbeef", 16).expect("valid hex");
```

```rust
assert_eq!(n, BigUint::from_u64(0xdead_beef));
assert_eq!(n.to_str_radix(16), "deadbeef");
assert_eq!(n.to_string(), "3735928559");
assert_eq!("3735928559".parse::<BigUint>(), Ok(n));

let value = BigUint::from_be_bytes(&[0x01, 0x00]);
assert_eq!(value, BigUint::from_u64(256));
assert_eq!(value.to_be_bytes_padded(4), vec![0x00, 0x00, 0x01, 0x00]);
```

2.2 Comparison and predicates

`BigUint` implements `Ord`: comparison walks limbs from the top and stops at the first difference. `bits` returns the significant bit count, $\lfloor \lg a \rfloor + 1$ for $a > 0$ and 0 for $a = 0$; `is_zero`, `is_one`, and `is_odd` are constant-limb predicates; `popcount` is the Hamming weight; `trailing_zeros` is the 2-adic valuation $v_2(a)$, `None` at zero.

2.3 Addition and subtraction

`add_ref` / `sub_ref` return new values; `add_assign_ref` / `sub_assign_ref` work in place; `assign_add` / `assign_sub` are three-operand forms whose output buffer is reused across calls (the shape of GMP's `mpz_add`). Both directions are the classical limb-wise carry and borrow chains. Subtraction panics on underflow: the type is unsigned, and a result that would be negative is a programming error — use `BigInt` where signs can go negative.

```rust
let a = BigUint::from_u64(1_000);
let b = BigUint::from_u64(37);
assert_eq!(a.add_ref(&b), BigUint::from_u64(1_037));
assert_eq!(a.sub_ref(&b), BigUint::from_u64(963));

// Three-operand form: ‘out‘’s storage is reused across calls.
let mut out = BigUint::zero();
out.assign_add(&a, &b);
assert_eq!(out, BigUint::from_u64(1_037));
```

2.4 Multiplication

`mul_ref` dispatches on operand size across a ladder of kernels, each asymptotically cheaper but with a larger constant, so each engages only past its measured crossover; `square_ref` delegates to `mul_ref`.

Schoolbook (Knuth, Algorithm M). The base case, $O(mn)$ limb products, used below 32 limbs.

Karatsuba (32 limbs and above). Split both operands at $B = \beta^s$, $a = a_1B + a_0$, $b = b_1B + b_0$. Three half-width products replace four:

$$\begin{aligned} z_0 &= a_0b_0, \qquad z_2 = a_1b_1, \\ z_1 &= (a_0 + a_1)(b_0 + b_1) - z_0 - z_2 \;=\; a_0b_1 + a_1b_0, \\ ab &= z_2B^2 + z_1B + z_0, \end{aligned} \tag{2}$$

giving $T(n) = 3\,T(n/2) + O(n) = O(n^{\lg 3}) \approx O(n^{1.585})$.

Toom–Cook 3 (128 limbs) and 4 (3072 limbs). Split each operand into k base-B digits and read it as a polynomial of degree $k-1$; the product polynomial has degree $2k-2$ and is determined by its values at $2k-1$ points. Toom-3 evaluates at $\{0, 1, -1, 2, \infty\}$ — five products of a third the size, $O(n^{\log_3 5}) \approx O(n^{1.465})$; Toom-4 at seven points, $O(n^{\log_4 7}) \approx O(n^{1.404})$. Interpolation is exact: its divisions by small constants leave no remainder.

Unbalanced operands (256 limbs). The balanced kernels all bound the length ratio of their operands. A lopsided product (long $\geq 2 \cdot$ short) whose shorter operand is past this kernel's own measured crossover — 256 limbs, far above Karatsuba's, because each block also pays dispatch and allocation overhead — is computed by block decomposition: with k the shorter length and $B = \beta^k$,

$$\text{long} \cdot \text{short} \;=\; \sum_i d_i \cdot \text{short} \cdot B^i, \qquad \text{long} = \sum_i d_iB^i,\ 0 \leq d_i < B, \tag{3}$$

a sum of balanced $k \times k$ products, each of which re-enters the dispatch and lands on Karatsuba or Toom, accumulated in place at its limb offset (shifting each product and adding at full width would cost $\Theta(\text{long}^2/k)$ limb copies and lose to flat schoolbook). Below the crossover the lopsided shapes stay schoolbook, which measurement favors there.

```
assert_eq!(a.mul_ref(&b), BigUint::from_u64(37_000));
assert_eq!(b.square_ref(), BigUint::from_u64(1_369));
```

2.5 Division and remainder

`div_rem` returns (q, r) with $a = qd + r$, $0 \leq r < d$, by Knuth's Algorithm D: normalize so the divisor's top limb is at least $\beta/2$, then estimate each quotient limb from the top two limbs of the running remainder against the top limb of the divisor,

$$\hat{q} \;=\; \min\!\left(\left\lfloor \frac{r_{j+n}\,\beta + r_{j+n-1}}{d_{n-1}} \right\rfloor,\ \beta - 1 \right), \tag{4}$$

which after the standard two-limb refinement is off by at most one, and the multiply-subtract step's borrow corrects that final case. `modulo` keeps only the remainder; `div_rem_u64` and `rem_u64` divide by a machine word with no heap-allocated divisor; `mod_mul` is one-shot modular multiplication. All panic on a zero divisor or modulus.

```
let n = BigUint::from_u64(1_000);
let d = BigUint::from_u64(7);
let (q, r) = n.div_rem(&d);
assert_eq!(q, BigUint::from_u64(142));
```

```
assert_eq!(r, BigUint::from_u64(6));
assert_eq!(n.rem_u64(7), 6);
```

2.6 Shifts and bit access

`shl_bits` / `shr_bits` shift by any count ($a \cdot 2^n$ and $\lfloor a/2^n \rfloor$); `shl1` / `shr1` are the single-bit forms the division kernel uses in its hot loop. `bit(i)` tests bit i, `set_bit(i)` sets it, and `bitxor_assign` is bitwise XOR — field addition in $\mathbb{F}_{2^m}$ (§6).

2.7 Integer roots

`sqrt_rem` returns $(r, a - r^2)$ for $r = \lfloor \sqrt{a} \rfloor$; `sqrt_floor` is the root alone and skips the final squaring that produces the remainder. Both run Newton's iteration on integer division,

$$x_{k+1} \;=\; \left\lfloor \frac{x_k + \lfloor a/x_k \rfloor}{2} \right\rfloor, \qquad x_0 = 2^{\lceil b/2 \rceil} \geq \lceil \sqrt{a}\, \rceil, \quad b = \lfloor \lg a \rfloor + 1. \tag{5}$$

By the AM–GM inequality $\frac{1}{2}(x + a/x) \geq \sqrt{a}$ for every $x > 0$, so every iterate stays at or above $\lfloor \sqrt{a} \rfloor$; from a seed above the root the sequence decreases strictly until it reaches it, and the first non-decrease certifies the answer (Cohen, Algorithm 1.7.1). Convergence is quadratic: about $\lg \lg a$ iterations, each one division.

`nth_root_floor(k)` generalizes with the same certificate:

$$x_{j+1} \;=\; \left\lfloor \frac{(k-1)\, x_j + \lfloor a/x_j^{k-1} \rfloor}{k} \right\rfloor. \tag{6}$$

`is_square` answers by residue filters — squares occupy 44 of the 256 residues modulo 256, and the single word modulus $9 \cdot 5 \cdot 7 \cdot 13 \cdot 17 = 69\,615$ folds five more character tests into one remainder — then one certified root for survivors. `is_perfect_power` tests $a = b^p$ for each prime $p \leq \lg a$ with one certified p-th root per exponent. `pow_u64` raises to a machine-word exponent by binary exponentiation.

```
let n = BigUint::from_u64(1_000_000);
let (root, rem) = n.sqrt_rem();
assert_eq!(root, BigUint::from_u64(1_000));
assert!(rem.is_zero());
assert!(n.is_square());
assert_eq!(n.nth_root_floor(3), BigUint::from_u64(100));
assert!(BigUint::from_u64(729).is_perfect_power()); // 3^6
```

2.8 Size estimates

`to_u64` narrows to a word when the value fits (`None` otherwise); `to_f64_lossy` saturates to infinity above the `f64` range; `ln_approx` computes $\ln a$ from the top bits and the bit length, staying finite far past the `f64` range — for size-driven parameter heuristics, not for arithmetic.

3 Signed integers: `BigInt` and `Sign`

A `BigInt` is a `Sign` (`Negative`, `Zero`, `Positive`) joined to a `BigUint` magnitude, with zero normalized to `Sign::Zero`. Construct with `from_biguint` (non-negative), `from_parts`, or `from_i64`; read back with `sign()` and `magnitude()`; `negated` flips the sign. `add_ref` / `sub_ref` are signed with in-place forms; `mul_ref` is the full signed product and `mul_biguint_ref` scales by an unsigned factor; `abs` returns the magnitude as a `BigUint`. `Ord` is sign-aware: negatives below zero below positives, magnitudes compared within a sign.

`div_rem` divides with remainder, *truncated toward zero* — the convention of C and of Rust's primitive `/` and `%`:

$$a \;=\; q\,d + r, \qquad |r| < |d|, \qquad \mathrm{sign}(r) \in \{\mathrm{sign}(a), 0\}, \tag{7}$$

so (-7).`div_rem`$(2) = (-3, -1)$ where floored division would give $(-4, 1)$. It panics on a zero divisor. `modulo_positive` is the floored counterpart against an unsigned modulus, mapping into the canonical range $[0, n)$:

$$a \bmod{}^{+} n \;=\; a - n \left\lfloor \frac{a}{n} \right\rfloor \in [0, n), \tag{8}$$

which extended Euclid and every modular routine here need (a negative representative would poison a Montgomery encoding).

```
let ten = BigInt::from_biguint(BigUint::from_u64(10));
let minus_three = BigInt::from_parts(Sign::Negative, BigUint::from_u64(3));
assert_eq!(ten.add_ref(&minus_three), BigInt::from_biguint(BigUint::from_u64(7)));

// The signed ring: full product, truncated division, absolute value.
assert_eq!(
    minus_three.mul_ref(&ten),
    BigInt::from_parts(Sign::Negative, BigUint::from_u64(30))
);
let minus_seven = BigInt::from_parts(Sign::Negative, BigUint::from_u64(7));
let two = BigInt::from_biguint(BigUint::from_u64(2));
let (q, r) = minus_seven.div_rem(&two);
assert_eq!(q, BigInt::from_parts(Sign::Negative, BigUint::from_u64(3)));
assert_eq!(r, BigInt::from_parts(Sign::Negative, BigUint::one()));
assert_eq!(minus_seven.abs(), BigUint::from_u64(7));

// -3 = 8 (mod 11), in canonical range.
assert_eq!(minus_three.modulo_positive(&BigUint::from_u64(11)), BigUint::from_u64(8)
    );
```

4 Barrett reduction: `BarrettCtx`

`BarrettCtx` is the fixed-modulus reduction context for a modulus of *either parity* — the complement to `MontgomeryCtx`, which requires an odd one. `BarrettCtx::new(n)` returns `None` for $n < 2$; otherwise it pays one division to precompute, for a k-limb modulus,

$$\mu \;=\; \left\lfloor \frac{\beta^{2k}}{n} \right\rfloor. \tag{9}$$

`reduce(x)`, for $0 \le x < \beta^{2k}$, estimates the quotient with two multiplications and no division (Handbook of Applied Cryptography, Algorithm 14.42):

$$\hat{q} \;=\; \left\lfloor \frac{\left\lfloor x/\beta^{\,k-1} \right\rfloor \cdot \mu}{\beta^{\,k+1}} \right\rfloor, \qquad q - 2 \;\le\; \hat{q} \;\le\; q \;=\; \left\lfloor \frac{x}{n} \right\rfloor, \tag{10}$$

so $x - \hat{q}\,n$ lies in $[0, 3n)$ and at most two conditional subtractions finish the reduction.

`mul_mod`, `square_mod`, and `pow_mod` are built on `reduce`. `pow_mod` is left-to-right binary exponentiation (Knuth, §4.6.3) with the accumulator seeded from the exponent's top set bit — one squaring per remaining exponent bit and one multiplication per remaining set bit, each followed by a reduction. $0^0 = 1$ by the usual convention.

```
let even = BigUint::from_u64(1_000);
let ctx = BarrettCtx::new(&even).expect("modulus is at least 2");
assert_eq!(
    ctx.mul_mod(&BigUint::from_u64(123), &BigUint::from_u64(456)),
    BigUint::from_u64(88) // 56 088 mod 1000
);
```

5 The Montgomery domain: `MontgomeryCtx`

Montgomery multiplication (Montgomery 1985) replaces division by a modulus n with division by a power of the limb base, which is a shift. Fix an odd modulus n of k limbs and let $R = \beta^{\,k}$, so $\gcd(R, n) = 1$. The *Montgomery representative* of x is

$$\bar{x} \;=\; xR \bmod n. \tag{11}$$

`MontgomeryCtx::new(n)` returns `None` for even or zero n; otherwise it precomputes $R^2 \bmod n$ (for encoding) and the word-level constant (Dussé and Kaliski, EUROCRYPT '90)

$$n'_0 \;=\; -\,n_0^{-1} \bmod \beta, \tag{12}$$

where n_0 is the low limb of n.

REDC. The core operation: given $0 \le T < nR$,

$$\begin{aligned} m \;&=\; \big((T \bmod R)\,n'\big) \bmod R, \qquad & n' &\equiv -n^{-1} \pmod{R}, \\ t \;&=\; \frac{T + m\,n}{R}, & t \;&\equiv\; TR^{-1} \pmod{n},\; 0 \le t < 2n, \end{aligned} \tag{13}$$

followed by one conditional subtraction. The division by R is exact by the choice of m ($T + mn \equiv 0 \bmod R$), and word-level interleaving needs only n'_0, never the full n'. Products in the domain multiply as

$$\text{mul\_mont}(\bar{x}, \bar{y}) \;=\; \text{REDC}(\bar{x}\,\bar{y}) \;=\; \overline{xy}, \tag{14}$$

so a long computation encodes once ($\bar{x} = \text{REDC}(x \cdot R^2)$), stays in the domain with `mul_mont` / `square_mont` / `add_mont` / `sub_mont` — the encoding is linear, so domain addition and subtraction are one compare-and-correct each — and decodes once at the boundary ($x = \text{REDC}(\bar{x})$). `one_mont()` is $\bar{1} = R \bmod n$.

`mul`, `square`, and `pow` are one-shot forms that convert internally; `pow_encoded` reuses an already encoded base across exponents. For loops where the product *is* the loop, `mul_mont_with_workspace` / `square_mont_with_workspace` thread one caller-owned scratch buffer through a sequence of domain operations, allocating it once instead of per multiply — measured per-operation at about 43% for a 64-bit modulus, roughly 25–33% at 256 bits, and roughly 20% at 512, falling to 2–3% at 2048 bits and to the edge of measurement (∼1%) at 4096; the in-tree `mont_workspace_timing` probe reproduces the numbers, printing its per-pass spread. The workspace holds unscrubbed intermediates, exactly as the discarded per-call buffer does. The exponentiation ladder has two engines, chosen by exponent width: above 64 bits, left-to-right k-ary with a fixed 4-bit window table held in the domain (HAC Algorithm 14.82), seeded from the top window; at 64 bits or fewer (e.g. the F4 public exponent 65 537), right-to-left binary square-and-multiply, which skips the window-table setup that would dominate so short a ladder. Both wipe their workspaces on exit.

Contract 5.1 (In-domain operands are reduced)**.** *`mul_mont` and `square_mont` (in both their allocating and `_with_workspace` forms) and `pow_encoded` require operands already reduced below the modulus. An operand* wider *than the modulus panics in every build (it does not fit the kernel's modulus-width windows); an unreduced operand of the modulus's width trips a debug assertion only, and in release yields a non-canonical result. The boundary operations `encode` / `decode` instead accept any representative and reduce.*

```rust
let p = BigUint::from_u64(97);
let ctx = MontgomeryCtx::new(&p).expect("97 is odd");
let a = BigUint::from_u64(5);
let b = BigUint::from_u64(6);

// One-shot operations convert in and out internally.
assert_eq!(ctx.mul(&a, &b), BigUint::from_u64(30));
assert_eq!(ctx.pow(&a, &BigUint::from_u64(3)), BigUint::from_u64(28)); // 125 mod 97

// Staying in the domain: encode once, multiply cheaply, decode once.
let a_mont = ctx.encode(&a);
let b_mont = ctx.encode(&b);
let product_mont = ctx.mul_mont(&a_mont, &b_mont);
assert_eq!(ctx.decode(&product_mont), BigUint::from_u64(30));

// Loops thread one workspace through the domain operations: the same
// values, one allocation instead of one per multiply.
let mut ws: Vec<u64> = Vec::new();
assert_eq!(ctx.mul_mont_with_workspace(&a_mont, &b_mont, &mut ws), product_mont);
assert_eq!(
    ctx.square_mont_with_workspace(&a_mont, &mut ws),
    ctx.square_mont(&a_mont)
);
assert_eq!(ctx.decode(&ctx.sub_mont(&a_mont, &b_mont)), BigUint::from_u64(96));
```

6 Binary fields: `Gf2m`

A `Gf2m` is the field $\mathbb{F}_{2^m} = \mathbb{F}_2[x]/(f)$ for an irreducible $f \in \mathbb{F}_2[x]$ of degree m. Field elements and the defining polynomial are `BigUint` bit patterns: bit i is the coefficient of x^i. `Gf2m::new(f)` derives $m = \deg f$ and returns `None` for constants; *irreducibility is the caller's contract* — `Gf2m::is_irreducible` (below) guards it for untrusted polynomials. On a reducible modulus the ring $\mathbb{F}_2[x]/(f)$ is not a field: `inverse` and `div` report non-units as `None` and `solve_quadratic` verifies its root, but `sqrt` silently returns a wrong value there, because squaring is a bijection only in a field.

6.1 Arithmetic

Addition is coefficient-wise XOR — `Gf2m::add` is an associated function, since it needs no modulus. Multiplication is the word-level comb with reduction (Guide to Elliptic Curve Cryptography, Algorithm 2.36); reduction folds one excess word at a time through the *tap identity*: writing $f = x^m + \sum_{t \in T} x^t$ with T the lower exponents,

$$x^m \equiv \sum_{t \in T} x^t \pmod{f}, \tag{15}$$

which is $\mathbb{F}_2$-linear and therefore applies to 64 coefficients at once — one shifted XOR per tap per word, four or fewer for the trinomial and pentanomial moduli every standard uses. Squaring is linear in characteristic 2,

$$\Big(\sum c_i x^i\Big)^2 = \sum c_i \, x^{2i}, \tag{16}$$

implemented by interleaving a zero bit after every coefficient bit through a spread table, then reducing. `inverse` is the extended Euclidean algorithm over $\mathbb{F}_2[x]$ (Guide to ECC, Algorithm 2.48), maintaining $u = b \cdot a + s \cdot f$ with only the cofactor of a tracked; `div(a, b)` is $a \cdot b^{-1}$, `None` for a non-unit divisor. `pow` is binary exponentiation over the field.

```
// GF(2^3) with x^3 + x + 1.
let field = Gf2m::new(BigUint::from_u64(0b1011)).expect("degree 3");
assert_eq!(field.degree(), 3);

// (x + 1)(x^2 + 1) = x^3 + x^2 + x + 1 = x^2 (mod f).
let x_plus_1 = BigUint::from_u64(0b011);
let x2_plus_1 = BigUint::from_u64(0b101);
assert_eq!(field.mul(&x_plus_1, &x2_plus_1), BigUint::from_u64(0b100));

// x * (x^2 + 1) = x^3 + x = 1, so they are inverses.
let x = BigUint::from_u64(0b010);
assert_eq!(field.inverse(&x), Some(x2_plus_1));
assert_eq!(field.inverse(&BigUint::zero()), None);
```

6.2 Trace, half-trace, square roots, and quadratics

The *absolute trace* is the $\mathbb{F}_2$-linear map

$$\operatorname{Tr}(c) = \sum_{i=0}^{m-1} c^{2^i} \in \mathbb{F}_2, \tag{17}$$

computed by $m-1$ squarings and XORs (`trace`). Because squaring is a field automorphism (Frobenius), every element has a unique square root,

$$\sqrt{c} \;=\; c^{\,2^{m-1}}, \tag{18}$$

computed by $m-1$ squarings (`sqrt`).

The quadratic $z^2 + z = c$ is solvable exactly when $\mathrm{Tr}(c) = 0$ (the map $z \mapsto z^2 + z$ is 2-to-1 onto the trace-zero hyperplane), and its two roots differ by 1. For odd m the *half-trace*

$$\mathrm{HT}(c) \;=\; \sum_{i=0}^{(m-1)/2} c^{\,2^{2i}} \qquad \text{satisfies} \qquad \mathrm{HT}(c)^2 + \mathrm{HT}(c) \;=\; c + \mathrm{Tr}(c), \tag{19}$$

so `half_trace` solves the quadratic directly when $\mathrm{Tr}(c) = 0$; it panics at even degree, where the identity collapses (every FIPS binary curve degree — 163, 233, 283, 409, 571 — is odd). `solve_quadratic` is the total form: it tests the trace, works at every degree (even degree via IEEE Std 1363-2000, Annex A.4.7), and verifies its root. Over an irreducible modulus it returns `None` exactly when $\mathrm{Tr}(c) = 1$; over a reducible one, also whenever the trace computation, the auxiliary trace-one element, or the final verification fails — the root-verifying design is what keeps a non-field from producing a silently wrong answer here.

```
// Tr(x) = 0 in GF(2^3), so the half-trace solves z^2 + z = x.
let z = field.half_trace(&x);
assert_eq!(Gf2m::add(&field.square(&z), &z), x);

// Squaring is a bijection; sqrt inverts it.
assert_eq!(field.sqrt(&BigUint::from_u64(0b100)), x);

// solve_quadratic is total across degrees - here in the AES byte field
// GF(2^8), where the degree is even and the half-trace does not apply.
let aes = Gf2m::new(BigUint::from_u64(0x11B)).expect("the AES byte field");
let a = BigUint::from_u64(0x53);
let c = Gf2m::add(&aes.square(&a), &a);
let z = aes.solve_quadratic(&c).expect("solvable by construction");
assert_eq!(Gf2m::add(&aes.square(&z), &z), c);
```

6.3 Irreducibility: Rabin's test

`Gf2m::is_irreducible(f)` decides irreducibility deterministically (Rabin 1980, the $\mathbb{F}_2$ form). The criterion rests on the identity that $x^{2^k} - x$ is the product of all monic irreducibles over $\mathbb{F}_2$ of degree dividing k: a polynomial f of degree m is irreducible if and only if

$$x^{2^m} \equiv x \pmod{f} \qquad \text{and} \qquad \gcd\!\big(x^{2^{m/q}} - x,\ f\big) = 1 \ \text{ for every prime } q \mid m. \tag{20}$$

The first condition confines every irreducible factor's degree to a divisor of m; the gcd conditions exclude the proper divisors (every proper divisor of m divides some m/q). Mechanically $x^{2^i} \bmod f$ is i applications of `square` to x, so one forward pass of m squarings visits the checkpoints m/q in ascending order and finishes at x^{2^m} — m squarings plus one polynomial gcd per distinct prime divisor of m.

```
assert!(Gf2m::is_irreducible(&BigUint::from_u64(0b1011)));  // x^3 + x + 1
assert!(!Gf2m::is_irreducible(&BigUint::from_u64(0b1001))); // x^3 + 1
assert!(Gf2m::is_irreducible(&BigUint::from_u64(0x11B)));   // AES
```

7 Number theory

7.1 Greatest common divisors

`gcd(a, b)` computes $\gcd(a, b)$ with $\gcd(a, 0) = a$; `lcm(a, b)` $= ab/\gcd(a, b)$, with $\mathrm{lcm}(a, 0) = 0$. `gcd_extended` returns the Bézout triple (g, s, t) of signed cofactors with

$$g \;=\; \gcd(a, b) \;=\; as + bt. \tag{21}$$

Three engines back these, chosen by size. Small operands run Euclid with word-level steps. Above that, Lehmer's algorithm (Knuth, §4.5.2, Algorithm L) simulates a whole run of Euclidean quotients on the leading words: each quotient extends a 2×2 integer transform

$$M \;=\; \begin{pmatrix} q & 1 \\ 1 & 0 \end{pmatrix}_k \cdots \begin{pmatrix} q & 1 \\ 1 & 0 \end{pmatrix}_1, \tag{22}$$

and the batch is certified against both endpoints of the leading-word interval before being replayed once, at full width, as a single 2×2 matrix–vector application (four scalar-by-bignum products) — roughly 35 quotients per 124-bit window. At very large sizes ($\sim$131 kbit, measured) `gcd` switches to the subquadratic Half-GCD (Möller 2008): recursively compute the transform that reduces the top halves, splice it onto the full operands, and recurse, for $T(n) = O(M(n) \log n)$ where M is the multiplication cost.

```
let a = BigUint::from_u64(240);
let b = BigUint::from_u64(46);
assert_eq!(gcd(&a, &b), BigUint::from_u64(2));
assert_eq!(
    lcm(&BigUint::from_u64(4), &BigUint::from_u64(6)),
    BigUint::from_u64(12)
);

let (g, s, t) = gcd_extended(&a, &b);
let bezout = s.mul_biguint_ref(&a).add_ref(&t.mul_biguint_ref(&b));
assert_eq!(bezout, BigInt::from_biguint(g));
```

7.2 Quadratic-residue symbols

For an odd prime p the Legendre symbol is

$$\left(\frac{a}{p}\right) \;=\; \begin{cases} 0 & p \mid a, \\ +1 & a \text{ is a non-zero square mod } p, \\ -1 & \text{otherwise,} \end{cases} \qquad \left(\frac{a}{p}\right) \equiv a^{(p-1)/2} \pmod{p}, \tag{23}$$

the congruence being Euler's criterion. The Jacobi symbol extends it multiplicatively to odd $n = \prod p_i^{e_i}$ as $\left(\frac{a}{n}\right) = \prod \left(\frac{a}{p_i}\right)^{e_i}$; `jacobi(a, n)` computes it without factoring n, by quadratic reciprocity (HAC Algorithm 2.149): for odd coprime a, n,

$$\left(\frac{a}{n}\right)\left(\frac{n}{a}\right) = (-1)^{\frac{a-1}{2}\cdot\frac{n-1}{2}}, \qquad \left(\frac{2}{n}\right) = (-1)^{\frac{n^2-1}{8}}, \tag{24}$$

alternating reductions $a \bmod n$ with extractions of 2 until the arguments collapse. (The arguments are unsigned, so the reduction never produces a negative top argument and the $\left(\frac{-1}{n}\right)$ supplement is never needed.) It returns `None` for even n, where the symbol is undefined; `legendre` is the same computation under its prime-modulus name; `kronecker` extends to every modulus through $\left(\frac{a}{2}\right) = 0$ for even a and $(-1)^{(a^2-1)/8}$ for odd, with $\left(\frac{a}{0}\right) = 1$ exactly when $a = 1$.

Note $\left(\frac{a}{n}\right) = +1$ does not certify that a is a residue for composite n — the two factors' symbols can both be -1. Only $\left(\frac{a}{n}\right) = -1$ certifies non-residuosity.

```
// (2/9) = 1 because 9 = 1 (mod 8); (3/9) = 0 by the shared factor.
assert_eq!(jacobi(&BigUint::from_u64(2), &BigUint::from_u64(9)), Some(1));
assert_eq!(jacobi(&BigUint::from_u64(3), &BigUint::from_u64(9)), Some(0));
// 2 is a residue mod 7 (3^2 = 9 = 2).
assert_eq!(legendre(&BigUint::from_u64(2), &BigUint::from_u64(7)), Some(1));
// Kronecker handles even moduli: (5/8) = (5/2)^3 = -1.
assert_eq!(kronecker(&BigUint::from_u64(5), &BigUint::from_u64(8)), -1);
```

7.3 Modular exponentiation and inverses

`mod_pow(b, e, n)` computes $b^e \bmod n$ for any $n \geq 1$: through the Montgomery ladder when n is odd, and by binary exponentiation with direct reductions when n is even (where Montgomery cannot operate — callers with a fixed even modulus should hold a `BarrettCtx`).

`mod_inverse(a, n)` returns $a^{-1} \bmod n$ from the extended Euclidean identity: when $\gcd(a, n) = 1$, the cofactor s in $as + nt = 1$ reduces to the inverse; when the gcd exceeds one there is no inverse and the result is `None`.

`mod_inverse_batch` inverts a whole slice at the cost of one inversion and $3(k-1)$ multiplications — Montgomery's trick. With prefix products $P_i = x_1 x_2 \cdots x_i$,

$$P_k^{-1} \text{ (one inversion)}, \qquad x_i^{-1} = P_{i-1} \cdot P_i^{-1}, \qquad P_{i-1}^{-1} = x_i \cdot P_i^{-1}, \tag{25}$$

walking the products back from the top. `None` when any element shares a factor with the modulus (which element is not identified — that would cost the inversions the trick avoids).

```
assert_eq!(
    mod_pow(&BigUint::from_u64(3), &BigUint::from_u64(4), &BigUint::from_u64(5)),
    BigUint::one() // 81 = 1 (mod 5)
);
assert_eq!(
    mod_inverse(&BigUint::from_u64(3), &BigUint::from_u64(7)),
    Some(BigUint::from_u64(5)) // 3 * 5 = 15 = 1 (mod 7)
);
assert_eq!(mod_inverse(&BigUint::from_u64(2), &BigUint::from_u64(4)), None);

let m = BigUint::from_u64(97);
let values = [BigUint::from_u64(3), BigUint::from_u64(10), BigUint::from_u64(96)];
let inverses = mod_inverse_batch(&values, &m).expect("all coprime to 97");
```

```
for (inv, v) in inverses.iter().zip(&values) {
    assert!(BigUint::mod_mul(inv, v, &m).is_one());
}
```

7.4 Square roots modulo a prime

`sqrt_mod(a, p)` returns a square root of a modulo an odd prime p, or `None` for a non-residue. Write $p-1 = 2^s q$ with q odd. Three engines serve the call, chosen by the 2-adic depth s.

The $p \equiv 3 \pmod 4$ shortcut. When $s = 1$ — half of all primes, so the branch that runs most often — the root is a single exponentiation: for a residue a,

$$r \;=\; a^{(p+1)/4} \pmod p, \qquad r^2 \;=\; a^{(p+1)/2} \;=\; a \cdot a^{(p-1)/2} \;=\; a, \tag{26}$$

the last step by Euler's criterion.

Tonelli–Shanks. For deeper s, initialize with a non-residue z (found by testing $\left(\frac{z}{p}\right) = -1$; $(p-1)/2$ of the non-zero residues qualify):

$$M = s, \quad c = z^q, \quad t = a^q, \quad r = a^{(q+1)/2} \pmod p. \tag{27}$$

The invariants $r^2 = a\,t$ and $\operatorname{ord}(t) \mid 2^{M-1}$ hold throughout; while $t \neq 1$, find the least i with $t^{2^i} = 1$, set $b = c^{2^{M-i-1}}$, and update

$$M \leftarrow i, \qquad c \leftarrow b^2, \qquad t \leftarrow tb^2, \qquad r \leftarrow rb. \tag{28}$$

Each pass strictly reduces M, so at most s passes run; when $t = 1$, $r^2 = a$.

Cipolla. When the 2-adic depth s is large the descent's $O(s^2)$ multiplications dominate, and the implementation dispatches to Cipolla's algorithm: find t with $\left(\frac{t^2-a}{p}\right) = -1$, work in $\mathbb{F}_{p^2} = \mathbb{F}_p(\omega)$ with $\omega^2 = t^2 - a$, and compute

$$r \;=\; (t+\omega)^{(p+1)/2}, \tag{29}$$

which lands in $\mathbb{F}_p$ and squares to a. Every path verifies its result by squaring before returning it, so on a composite modulus — for which the internal reasoning is unsound — the answer is either `None` or a genuine square root modulo that composite, never a silently wrong value. `None` is *not* a compositeness certificate, and `Some` is not a primality one: `sqrt_mod`$(1, 15)$ returns `Some(1)`.

```
let p = BigUint::from_u64(41);
let root = sqrt_mod(&BigUint::from_u64(2), &p).expect("2 is a residue mod 41");
assert_eq!(BigUint::mod_mul(&root, &root, &p), BigUint::from_u64(2));
assert_eq!(sqrt_mod(&BigUint::from_u64(3), &p), None); // non-residue
```

7.5 Square roots modulo a prime power

`sqrt_mod_prime_power(a, p, e)` returns *every* square root of a modulo p^e, ascending, empty for a non-residue. For odd p and $p \nmid a$, a root mod p lifts uniquely along Hensel's construction: given $r_k^2 \equiv a \pmod{p^k}$,

$$r_{k+1} \;=\; r_k + t\,p^k, \qquad t \;=\; \frac{a - r_k^2}{p^k} \cdot (2r_k)^{-1} \bmod p, \tag{30}$$

and the two roots mod p give the two mod p^e. For $p = 2$ the structure is governed by the residue of a modulo 8 (one root mod 2, two mod 4, and four mod 2^e for $e \geq 3$ when $a \equiv 1 \bmod 8$). For $p \mid a$, factor out the largest even power p^{2j} dividing a and recurse on the cofactor, multiplying the roots back by p^j and expanding the residual multiplicity. Panics when $e = 0$ or $p < 2$.

```
// 9 has four square roots mod 16: 3, 5, 11, 13.
let roots = sqrt_mod_prime_power(&BigUint::from_u64(9), &BigUint::from_u64(2), 4);
assert_eq!(roots, vec![
    BigUint::from_u64(3), BigUint::from_u64(5),
    BigUint::from_u64(11), BigUint::from_u64(13),
]);
```

7.6 The Chinese Remainder Theorem

`crt_combine` solves the simultaneous congruences $x \equiv a_i \pmod{m_i}$ for pairwise coprime moduli: the unique x mod M, $M = \prod m_i$, assembled by folding pairs,

$$x_{12} \;=\; a_1 + m_1\Big((a_2 - a_1)\, m_1^{-1} \bmod m_2\Big) \;\equiv\; a_i \pmod{m_i},\; i = 1, 2. \tag{31}$$

`None` when the moduli are not pairwise coprime (detected by the failing inversion).

```
// Sunzi's classic: 2 mod 3, 3 mod 5, 2 mod 7.
let x = crt_combine(&[
    (BigUint::from_u64(2), BigUint::from_u64(3)),
    (BigUint::from_u64(3), BigUint::from_u64(5)),
    (BigUint::from_u64(2), BigUint::from_u64(7)),
])
.expect("moduli are pairwise coprime");
assert_eq!(x, BigUint::from_u64(23));
```

7.7 Valuation

`valuation(n, p)` is the p-adic valuation $v_p(n)$, the exponent of p in n; `remove_factor(n, p)` returns $(n/p^{v_p(n)},\, v_p(n))$, dividing through a ladder of squared powers $p, p^2, p^4, \ldots$ so a valuation of v costs $O(\lg v)$ divisions (the shape of GMP's `mpz_remove`). Both panic on $n = 0$ (the valuation is unbounded) or $p < 2$.

```
let n = BigUint::from_u64(3_888); // 2^4 * 3^5
assert_eq!(valuation(&n, &BigUint::from_u64(2)), 4);
let (cofactor, exponent) = remove_factor(&n, &BigUint::from_u64(3));
assert_eq!(exponent, 5);
assert_eq!(cofactor, BigUint::from_u64(16));
```

7.8 Rational reconstruction

Modular and p-adic computation ends by reading a rational answer back from its residue. Given x and m, `rational_reconstruct_bounded` finds the unique fraction p/q with

$$p \equiv q\,x \pmod{m}, \qquad |p| \le N, \quad 0 < q \le D, \quad \gcd(p, q) = 1, \tag{32}$$

under the caller's contract $2ND < m$, which is precisely what makes the answer unique (the difference of two candidate fractions would be a non-zero multiple of m over a denominator smaller than m). The contract is enforced: a call with $2ND \ge m$ *panics*, since any answer it produced could be one of several. The candidate comes from the extended Euclidean remainder sequence of (m, x), stopped at the first remainder $r \le N$: writing $r \equiv t\,x \pmod{m}$ for that row's cofactor t, the only possible answer is $p = \pm r$, $q = |t|$ (Wang 1981; von zur Gathen and Gerhard, §5.10). The final checks $q \le D$ and $\gcd(r, t) = 1$ decide existence.

`rational_reconstruct` is the symmetric special case $N = D = \lfloor\sqrt{(m-1)/2}\rfloor$. When reconstructing many values under one modulus, compute that bound once and call the bounded form — the wrapper's square root costs more than the reconstruction itself at large sizes.

```
// 22/7 survives reduction mod 1009 and comes back intact.
let m = BigUint::from_u64(1_009);
let seven_inv = mod_inverse(&BigUint::from_u64(7), &m).expect("7 is invertible");
let x = BigUint::mod_mul(&BigUint::from_u64(22), &seven_inv, &m);
let (p, q) = rational_reconstruct(&x, &m).expect("22/7 is within the bounds");
assert_eq!(p, BigInt::from_biguint(BigUint::from_u64(22)));
assert_eq!(q, BigUint::from_u64(7));

// The bounded form makes the contract explicit (here N = 22, D = 7).
assert_eq!(
    rational_reconstruct_bounded(&x, &m, &BigUint::from_u64(22), &BigUint::from_u64
    (7)),
    Some((BigInt::from_biguint(BigUint::from_u64(22)), BigUint::from_u64(7)))
);
```

7.9 Product trees, remainder trees, and batch smoothness

`product_tree(values)` builds the pairwise-product tree over its leaves — level 0 the values, each parent the product of its two children, the root $\prod_i x_i$. `remainder_tree(tree, z)` reduces z down the tree, $z \bmod$ each node, reaching $z \bmod x_i$ at every leaf for one tree descent instead of k full-width divisions; it panics on a zero leaf, which it would divide by.

`smooth_parts(values, primes)` is Bernstein's batched smoothness test. With $z = \prod_{p \in P} p$ (itself built by a product tree) and $r_i = z \bmod x_i$ from the remainder tree, square repeatedly,

$$s_i = r_i^{2^{e_i}} \bmod x_i, \qquad e_i = \lfloor \lg b_i \rfloor + 1 \text{ squarings for } b_i = \lfloor \lg x_i \rfloor + 1, \text{ so } 2^{e_i} > b_i \ge \lg x_i, \tag{33}$$

which pushes every smooth prime's multiplicity in s_i past its multiplicity in x_i (no prime can divide x_i more than $\lg x_i$ times). Then

$$\gcd(x_i, s_i) = \text{the largest divisor of } x_i \text{ composed of primes in } P, \tag{34}$$

the *smooth part*, with full multiplicity. A value is fully smooth exactly when its smooth part equals itself; $0 \mapsto 0$ and $1 \mapsto 1$ pass through without joining the tree.

```
let primes = primes_below(10); // 2, 3, 5, 7
let values = [
    BigUint::from_u64(360),    // 2^3 * 3^2 * 5 - fully smooth
    BigUint::from_u64(2 * 11), // 11 is not in the base
];
let parts = smooth_parts(&values, &primes);
assert_eq!(parts[0], BigUint::from_u64(360));
assert_eq!(parts[1], BigUint::from_u64(2));

// The trees are public on their own: the root is the product, and one
// descent reduces a modulus against every leaf.
let leaves = [BigUint::from_u64(7), BigUint::from_u64(11), BigUint::from_u64(13)];
let tree = product_tree(&leaves);
assert_eq!(tree.last().unwrap()[0], BigUint::from_u64(7 * 11 * 13)); // 1001
let residues = remainder_tree(&tree, &BigUint::from_u64(100));
assert_eq!(residues, vec![
    BigUint::from_u64(100 % 7),
    BigUint::from_u64(100 % 11),
    BigUint::from_u64(100 % 13),
]);
```

`primes_below(bound)` sieves every prime below the bound into a `Vec<u64>` — the bulk companion to the probabilistic tests below.

7.10 Primality

Miller–Rabin. For odd $n > 2$ write $n - 1 = 2^s d$ with d odd. A base $a \not\equiv 0, \pm 1 \pmod{n}$ is a *witness* to compositeness unless

$$a^d \equiv 1 \pmod{n} \qquad \text{or} \qquad a^{2^r d} \equiv -1 \pmod{n} \text{ for some } 0 \le r < s. \tag{35}$$

Every prime satisfies one of the two for every base; for a composite, at least three quarters of the bases are witnesses, so k random rounds err with probability at most 4^{-k}. `miller_rabin_witness(n, a)` is the single-round primitive: `true` means a proves n composite.

`is_probable_prime` runs trial division by small primes, then Miller–Rabin over twelve fixed small-prime bases (2 through 37) — a deterministic proof below $\psi_{12} \approx 3.19 \times 10^{23}$ (exactly 318 665 857 834 031 151 167 461: Sorenson and Webster 2017; the often-quoted 3.317×10^{24} is ψ_{13} and requires a thirteenth base) and a strong probable-prime test above. `is_probable_prime_with_bases` takes an explicit base set after the same unconditional sieve; each base is reduced modulo the candidate, trivial residues $\{0, 1, n-1\}$ are discarded, and a set with no effective round proves nothing and reports composite. Fixed bases are for candidates you generated yourself: an adversary can construct pseudoprimes against any fixed base set, so untrusted input needs candidate-derived witnesses added on top.

The strong Lucas test and Baillie–PSW. Selfridge's Method A picks the discriminant D as the first of $5, -7, 9, -11, 13, \ldots$ (the integers $\equiv 1 \bmod 4$ with $|D| \ge 5$, ordered by absolute value) with $\left(\frac{D}{n}\right) = -1$, then sets $P = 1$, $Q = (1 - D)/4$. The search itself can prove n composite and stop: a zero symbol whose gcd with n is proper exposes a factor, and a perfect square — for which no D has symbol -1 and the search would never end — is detected by one integer square root after the third failed candidate and likewise reported composite (Baillie and Wagstaff, §6). The Lucas sequences

$$U_0 = 0,\ U_1 = 1,\ V_0 = 2,\ V_1 = P, \qquad U_{k+1} = P\,U_k - Q\,U_{k-1}, \quad V_{k+1} = P\,V_k - Q\,V_{k-1}, \tag{36}$$

are computed modulo n by the doubling and stepping identities

$$U_{2k} = U_k V_k, \quad V_{2k} = V_k^2 - 2Q^k, \quad U_{k+1} = \tfrac{PU_k + V_k}{2}, \quad V_{k+1} = \tfrac{DU_k + PV_k}{2}, \tag{37}$$

the halvings exact modulo odd n. Writing $n + 1 = 2^s d$ with d odd, n is a *strong Lucas probable prime* when

$$U_d \equiv 0 \pmod{n} \qquad \text{or} \qquad V_{2^r d} \equiv 0 \pmod{n} \text{ for some } 0 \le r < s, \tag{38}$$

conditions every prime with $\left(\frac{D}{n}\right) = -1$ satisfies (Baillie and Wagstaff 1980). This stage alone is exposed as `is_strong_lucas_probable_prime`.

`is_probable_prime_bpsw` is the Baillie–PSW composite of the two: trial division, one strong base-2 Miller–Rabin round, then the strong Lucas test. The two stages fail on disjoint kinds of composites as far as anyone has found: no composite passing both is known, and none exists below 2^{64}, where the test is therefore deterministic.

```
assert!(is_probable_prime(&BigUint::from_u64(65_537)));
assert!(!is_probable_prime(&BigUint::from_u64(561))); // Carmichael

// 2047 = 23 * 89 is the smallest strong pseudoprime to base 2.
let n = BigUint::from_u64(2_047);
assert!(!miller_rabin_witness(&n, &BigUint::from_u64(2))); // 2 is fooled
assert!(miller_rabin_witness(&n, &BigUint::from_u64(3)));  // 3 is a witness

// with_bases reduces each base modulo n and discards {0, 1, n-1}; a set
// of only trivial bases runs no effective round and is not reported prime.
let composite = BigUint::from_u64(1_022_117); // 1009 x 1013, survives the sieve
assert!(!is_probable_prime_with_bases(&composite, &[1])); // 1 never testifies
assert!(!is_probable_prime_with_bases(&composite, &[2])); // 2 exposes it

// BPSW: the stages reject disjoint composites.
assert!(!is_probable_prime_bpsw(&BigUint::from_u64(2_047))); // Lucas rejects
assert!(is_strong_lucas_probable_prime(&BigUint::from_u64(5_459)));
assert!(!is_probable_prime_bpsw(&BigUint::from_u64(5_459))); // base 2 rejects
```

8 Polynomials: `PolyZ` and `PolyModP`

Both types are dense univariate polynomials, coefficients stored low to high and normalized to drop trailing zeros, so the zero polynomial is the empty coefficient list and `degree()` is `None` for it. `PolyZ` works over $\mathbb{Z}$; `PolyModP` over $\mathbb{Z}/m\mathbb{Z}$ with every coefficient held as its least non-negative residue and the modulus carried by the value — mixing two moduli in one operation is a hard panic in every build. The division-based operations of `PolyModP` require a *prime* modulus (they invert leading coefficients); a composite modulus panics on the first non-invertible pivot.

8.1 Arithmetic over $\mathbb{Z}$

`add`/`sub`/`mul` (schoolbook convolution), `evaluate` (Horner: fold $acc \leftarrow acc \cdot x + c_i$ from the top — one multiplication and addition per coefficient), `derivative` (the formal rule $\sum i\, c_i x^{i-1}$), `negated`, and `scale`. `content` is the gcd of the coefficients; `primitive_part` divides it out, by Gauss's lemma the canonical split $f = \mathrm{cont}(f) \cdot \mathrm{pp}(f)$.

8.2 Division over $\mathbb{Z}$

$\mathbb{Z}$ is not a field, so two divisions exist.

Exact division. `div_rem` returns (q, r) with $f = q\,g + r$, $\deg r < \deg g$, when an integer quotient exists, and `None` otherwise: each long-division step must divide the running remainder's leading coefficient exactly by $\mathrm{lc}(g)$ (always possible for monic g). The quotient is unique when it exists, so a single failing step is conclusive.

Pseudo-division. `pseudo_div_rem` premultiplies instead of dividing (Knuth, Algorithm R), staying in $\mathbb{Z}$ unconditionally:

$$\ell\, f \;=\; q\, g + r, \qquad \deg r < \deg g, \qquad \ell = \mathrm{lc}(g)^{\deg f - \deg g + 1} \tag{39}$$

(and $\ell = 1$ when $\deg f < \deg g$, where the result is $(0, f)$).

```
// (x + 1)(x + 2) = x^2 + 3x + 2, over Z.
let a = PolyZ::from_i64_slice(&[1, 1]); // x + 1
let b = PolyZ::from_i64_slice(&[2, 1]); // x + 2
assert_eq!(a.mul(&b), PolyZ::from_i64_slice(&[2, 3, 1]));

// Exact division: (x^2 + 3x + 2) / (x + 1) = x + 2, no remainder.
let (q, r) = a.mul(&b).div_rem(&a).expect("x + 1 divides");
assert_eq!(q, b);
assert!(r.is_zero());
```

8.3 Resultants and discriminants

The resultant of $f, g \in \mathbb{Z}[x]$ with roots α_i, β_j (over $\overline{\mathbb{Q}}$) is

$$\mathrm{Res}(f, g) \;=\; \mathrm{lc}(f)^{\deg g}\; \mathrm{lc}(g)^{\deg f} \prod_{i,j} (\alpha_i - \beta_j) \;=\; \det S(f, g), \tag{40}$$

the determinant of the Sylvester matrix — zero exactly when f and g share a non-constant factor. `resultant` computes it by Bareiss fraction-free elimination, which keeps every intermediate entry an integer minor determinant, so no rational arithmetic appears. In the number field sieve, Res *is* the algebraic norm.

The discriminant is

$$\mathrm{disc}(f) \;=\; \frac{(-1)^{d(d-1)/2}}{\mathrm{lc}(f)}\, \mathrm{Res}(f, f'), \qquad d = \deg f, \tag{41}$$

zero exactly when f has a repeated factor; the division by $\mathrm{lc}(f)$ is exact over $\mathbb{Z}$.

```
// disc(x^2 + 5x + 6) = 25 - 24 = 1.
assert_eq!(PolyZ::from_i64_slice(&[6, 5, 1]).discriminant(), BigInt::from_i64(1));
// (x-1)(x-2) and (x-1)(x-3) share (x-1): resultant zero.
let u = PolyZ::from_i64_slice(&[2, -3, 1]);
let v = PolyZ::from_i64_slice(&[3, -4, 1]);
assert_eq!(u.resultant(&v), BigInt::zero());
```

8.4 Arithmetic over $\mathbb{F}_p$

`PolyModP` adds the field-division layer: `div_rem` and `rem` (schoolbook long division with the one leading-coefficient inverse hoisted out of the loop — and skipped entirely for a monic divisor; `rem` runs the same loop without quotient bookkeeping, the form the gcd descent wants), `gcd` (Euclid on remainders, result returned monic — over a field the gcd is unique only up to a unit), `make_monic`, and `pow_mod` (binary exponentiation with a polynomial reduction after every step, the primitive behind the Frobenius computations below).

```
// Over Z/7Z: (x - 1)(x - 2) and (x - 2)(x - 3) share x - 2.
let p = BigUint::from_u64(7);
let f = PolyModP::from_poly_z(&PolyZ::from_i64_slice(&[2, -3, 1]), &p);
let g = PolyModP::from_poly_z(&PolyZ::from_i64_slice(&[6, -5, 1]), &p);
let shared = PolyModP::from_poly_z(&PolyZ::from_i64_slice(&[-2, 1]), &p);
assert_eq!(f.gcd(&g), shared);
```

8.5 Factorization over $\mathbb{F}_p$

`factor` returns the monic irreducible factors with multiplicities; it composes three classical stages.

Squarefree factorization. Write $f = \prod a_i^i$ with the a_i squarefree and pairwise coprime. Because $\frac{d}{dx}a^i = i\, a^{i-1}a'$ vanishes exactly when $p \mid i$,

$$\gcd(f, f') \;=\; \prod_{p\nmid i} a_i^{i-1} \cdot \prod_{p\,|\,i} a_i^{i}, \tag{42}$$

so $f/\gcd(f, f')$ is the product of the distinct factors of multiplicity prime to p, and a gcd cascade peels them off one multiplicity at a time. What remains is a p-th power, and in $\mathbb{F}_p[x]$,

$$g(x)^p \;=\; g(x^p), \tag{43}$$

so its p-th root is read off by taking every p-th coefficient (Frobenius fixes $\mathbb{F}_p$), and the recursion continues with multiplicities scaled by p. `squarefree_factorization` exposes this stage.

Distinct-degree factorization. The key identity: over $\mathbb{F}_p$,

$$x^{p^d} - x \;=\; \prod_{\substack{h \text{ monic irreducible} \\ \deg h\,|\,d}} h(x), \tag{44}$$

so $\gcd(f,\ x^{p^d} - x)$ extracts from a squarefree f the product of its irreducible factors of degree dividing d. Computing $x^{p^d} \bmod f$ is d Frobenius steps by `pow_mod`; sweeping $d = 1, 2, \ldots$ splits f into blocks of equal-degree factors.

Equal-degree splitting (Cantor–Zassenhaus). A block f of $k > 1$ irreducible factors, all of degree d, splits probabilistically: for a uniform random h with $\deg h < \deg f$ and odd p,

$$\gcd\Big(h^{(p^d-1)/2} - 1,\ f\Big) \tag{45}$$

is a proper factor with probability at least $\frac{1}{2}$ per pair of factors, because modulo each irreducible factor h lands in $\mathbb{F}_{p^d}^{\times}$ and $h^{(p^d-1)/2} = \pm 1$ independently, each sign with probability near $\frac{1}{2}$. For $p = 2$, where $(p^d - 1)/2$ is not an integer, the splitter is the trace map $T(h) = h + h^2 + h^4 + \cdots + h^{2^{d-1}}$ instead, which takes each factor to $\mathbb{F}_2$ with the same independent coin-flip behaviour. The routine is Las Vegas — retries never return a wrong split — and caps consecutive fruitless draws so a broken generator fails loudly rather than spinning.

`is_irreducible` runs the distinct-degree sieve as a test; `roots(rng)` extracts the linear factors of $\gcd(f, x^p - x)$ and returns the residues where f vanishes. Factoring and root-finding take an `Rng` (§10).

```
struct Lcg(u64);
impl Rng for Lcg {
    fn fill_bytes(&mut self, d: &mut [u8]) {
        for b in d {
            self.0 = self.0.wrapping_mul(6364136223846793005).wrapping_add(1);
            *b = (self.0 >> 33) as u8;
        }
    }
}
let p = BigUint::from_u64(7);
let mut rng = Lcg(0x1234_5678);

// x^2 - 1 = (x - 1)(x + 1) mod 7 - two roots.
let f = PolyModP::from_poly_z(&PolyZ::from_i64_slice(&[-1, 0, 1]), &p);
let mut roots = f.roots(&mut rng);
roots.sort();
assert_eq!(roots, vec![BigUint::from_u64(1), BigUint::from_u64(6)]);

// x^2 + 1 is irreducible mod 7 (-1 is a non-residue), so no roots.
let g = PolyModP::from_poly_z(&PolyZ::from_i64_slice(&[1, 0, 1]), &p);
assert!(g.is_irreducible());
assert!(g.roots(&mut rng).is_empty());

// factor returns monic irreducibles with multiplicities:
// x^3 + x = x * (x^2 + 1) over F_7.
let h = PolyModP::from_poly_z(&PolyZ::from_i64_slice(&[0, 1, 0, 1]), &p);
let mut fs = h.factor(&mut rng);
fs.sort_by_key(|(fac, _)| fac.degree());
assert_eq!(fs.len(), 2);
```

9 Lattice reduction: `lll_reduce`

`lll_reduce(basis)` applies the Lenstra–Lenstra–Lovász algorithm to an ordered lattice basis — the rows $b_1, \ldots, b_n$ of a `&mut [Vec<BigInt>]` — replacing it in place with a short, nearly orthogonal basis of the same lattice. The rows must be linearly independent.

With Gram–Schmidt vectors and coefficients

$$b_i^* \;=\; b_i - \sum_{j<i} \mu_{ij}\, b_j^*, \qquad \mu_{ij} \;=\; \frac{\langle b_i, b_j^*\rangle}{\langle b_j^*, b_j^*\rangle}, \tag{46}$$

a basis is *LLL-reduced* with parameter δ when

$$|\mu_{ij}| \le \tfrac{1}{2} \;\; (j < i) \qquad \text{and} \qquad (\delta - \mu_{i,i-1}^2)\|b_{i-1}^*\|^2 \;\le\; \|b_i^*\|^2 \quad \text{(Lovász condition).} \tag{47}$$

For $\delta = \frac{3}{4}$ (the default; `lll_reduce_delta` takes any fraction in $(\frac{1}{4}, 1)$) the first vector of a reduced basis satisfies

$$\|b_1\| \;\le\; 2^{(n-1)/4}(\det L)^{1/n}, \qquad \|b_1\| \;\le\; 2^{(n-1)/2}\,\lambda_1(L), \tag{48}$$

where $\lambda_1(L)$ is the length of a shortest non-zero lattice vector. The implementation is the *integral* variant (Cohen, Algorithm 2.6.3): it carries the Gram–Schmidt data as exact integers $d_j = \prod_{i \le j}\|b_i^*\|^2$ and $\lambda_{ij} = d_j\,\mu_{ij}$, so no rational or floating-point arithmetic appears and the output is exact. An independent rational-LLL oracle in the test suite pins its behavior.

```
let row = |xs: &[i64]| xs.iter().map(|&x| BigInt::from_i64(x)).collect::<Vec<_>>();
// A badly skewed basis for a 2-D lattice.
let mut basis = vec![row(&[201, 37]), row(&[1648, 297])];
lll_reduce(&mut basis);
// Reduction returns short vectors spanning the same lattice.
assert_eq!(basis, vec![row(&[1, 32]), row(&[40, 1])]);

// An explicit Lovasz parameter, as the fraction 99/100.
let mut basis = vec![row(&[201, 37]), row(&[1648, 297])];
lll_reduce_delta(&mut basis, 99, 100);
assert_eq!(basis, vec![row(&[1, 32]), row(&[40, 1])]);
```

10 Random sampling

rump chooses no entropy source. Every sampler is driven by a caller-supplied generator implementing the one-method trait

```
pub trait Rng {
    fn fill_bytes(&mut self, dest: &mut [u8]);
}
```

and the output is exactly as good as that source — cryptographic callers must supply a CSPRNG.

Every sampler is a *rejection* sampler, which buys exact uniformity: reducing a wide draw modulo the bound would over-represent the low residues whenever the bound is not a power of two. `random_below` draws uniformly from the enclosing power-of-two range $[0, 2^b)$, b the bound's bit length, and rejects draws at or above the bound; since the bound is at least 2^{b-1}, each draw is accepted with probability at least $\frac{1}{2}$ and the expected number of draws is at most 2. `random_nonzero_below` additionally rejects zero; `random_coprime_below(rng, n, m)` keeps the first draw coprime to m — pass n as both bound and modulus to draw a uniform unit of $(\mathbb{Z}/n\mathbb{Z})^\times$. `random_probable_prime(rng, bits)` draws odd candidates of exactly the requested width (top and low bits forced) and screens them with `is_probable_prime`; by the prime number theorem the expected number of rounds is about $0.35 \cdot \text{bits}$.

Degenerate *arguments* return `None` (an empty range, a zero coprimality modulus, a width below 2). For a degenerate *generator*, each sampler carries a stall guard that panics rather than loop forever on the sources it can soundly detect, every guard sized so a working generator trips it with probability at most $e^{-111} \approx 2^{-160}$, the loosest of the individual bounds (each bullet gives its own). Their coverage differs, and the one gap is stated rather than papered over:

- `random_below` and `random_nonzero_below` accept each draw with probability at least $\frac{1}{2}$ regardless of arguments, so they bound consecutive *rejections* at 256 (survival probability at most 2^{-256}) and catch every degenerate source.
- `random_probable_prime`'s acceptance density is a function of the width alone, so its rejection bound scales with the width: $64 \cdot \text{bits}$ fruitless rounds, which a working generator survives with probability $(1 - \frac{2}{\text{bits} \cdot \ln 2})^{64 \cdot \text{bits}} \approx e^{-185}$ by the asymptotic prime density, and below e^{-111} unconditionally at every width: for bits ≥ 6 from the lower bound $\pi(2x) - \pi(x) > \frac{3}{5}\, x / \ln x$ (Rosser–Schoenfeld 1962), and for the four smaller widths by exhaustive enumeration (widths 2 and 3 contain no composite; the worst case is width 4, density $\frac{1}{2}$ over 256 rounds, survival 2^{-256}) — so it, too, catches every degenerate source. A *constant* generator additionally fails fast: a candidate equal to the previous rejected one is known composite, skips the Miller–Rabin re-screen, and trips a 256-repeat assertion — one screen and 256 comparisons, not hours of full-width exponentiation at large widths.
- `random_coprime_below` is the one sampler where no *usable* rejection count exists. A primorial modulus legitimately leaves 1 as the only unit below the bound, so acceptance can be as thin as $1/(\mathit{upper} - 1)$; the only sound count therefore scales with the bound itself ($\Theta(\mathit{upper})$ draws), unreachable for a multiprecision bound. Its guard instead detects a *pinned* generator (the same rejected candidate 256 times running, probability at most 2^{-256} for a working source). A degenerate source that cycles among several rejected values is statistically indistinguishable from an unlucky legitimate run against a sparse unit set, does not trip the guard, and remains the caller's to avoid.

```rust
/// xorshift64: deterministic and compact. Fine for a manual; NOT a CSPRNG.
struct XorShift64(u64);

impl Rng for XorShift64 {
    fn fill_bytes(&mut self, dest: &mut [u8]) {
        for chunk in dest.chunks_mut(8) {
            self.0 ^= self.0 << 13;
            self.0 ^= self.0 >> 7;
            self.0 ^= self.0 << 17;
            let word = self.0.to_le_bytes();
            chunk.copy_from_slice(&word[..chunk.len()]);
        }
    }
}

let mut rng = XorShift64(0x1234_5678_9abc_def0);
let bound = BigUint::from_u64(1_000_003);
```

```rust
let draw = random_below(&mut rng, &bound).expect("bound is non-zero");
assert!(draw < bound);

let nonzero = random_nonzero_below(&mut rng, &bound).expect("bound exceeds one");
assert!(!nonzero.is_zero() && nonzero < bound);

let unit = random_coprime_below(&mut rng, &bound, &BigUint::from_u64(30_030))
    .expect("units exist below the bound");
assert_eq!(gcd(&unit, &BigUint::from_u64(30_030)), BigUint::one());

let prime = random_probable_prime(&mut rng, 64).expect("width of at least 2");
assert_eq!(prime.bits(), 64);
assert!(is_probable_prime(&prime));
```

11 Panics

The unsigned types treat impossible requests as programming errors, not recoverable conditions. Fallible *mathematics* — a missing inverse, a non-residue, an even Montgomery modulus, non-coprime CRT moduli — returns `Option` instead.

Call	Panics when
`sub_ref` / `sub_assign_ref`	the result would be negative
`div_rem` / `div_rem_u64` / `modulo` / `rem_u64`	the divisor or modulus is zero
`mod_mul` / `mod_pow` / `mod_add` / `mod_sub`	the modulus is zero
`ln_approx`	the value is zero
`nth_root_floor`	$k = 0$
`sqrt_mod_prime_power`	$e = 0$ or $p < 2$
`valuation` / `remove_factor`	$n = 0$ or $p < 2$
`remainder_tree`	a leaf is zero
`rational_reconstruct_bounded`	$2ND \geq m$ (the uniqueness contract)
`to_be_bytes_padded`	the value exceeds the requested width
`from_str_radix` / `to_str_radix`	the radix is outside $[2, 36]$
`mul_mont` / `square_mont` / their `_with_workspace` forms / `pow_encoded`	an operand *wider* than the modulus, in every build; an unreduced operand of the modulus's width trips a debug assert only (§5)
`Gf2m::half_trace`	the field degree is even
`PolyZ` / `PolyModP` division	the divisor is the zero polynomial
`PolyModP::new` / `zero` / `from_poly_z`	the modulus is below 2
`PolyModP` (mixed moduli)	the two operands carry different moduli
`PolyModP` division / `gcd` / `factor`	a non-invertible pivot (composite modulus)
`squarefree_factorization` / `factor`	the polynomial is constant or zero
`factor` / `roots`	the supplied `Rng` makes no progress (the equal-degree splitter's stall guard)
`lll_reduce` / `lll_reduce_delta`	dependent rows, ragged or zero-length rows; the `_delta` form also on $\delta \notin (\frac{1}{4}, 1)$ or a zero denominator
samplers (§10)	the generator trips a stall guard (see §10 for what each guard can and cannot detect)

References

- D. E. Knuth, *The Art of Computer Programming*, vol. 2: *Seminumerical Algorithms*, 3rd ed. — Algorithms M, D, L; §4.3.3 (Karatsuba, Toom–Cook); §4.6.3 (exponentiation).

- P. L. Montgomery, *Modular multiplication without trial division*, Math. Comp. 44 (1985).

- S. R. Dussé and B. S. Kaliski, *A cryptographic library for the Motorola DSP56000*, EUROCRYPT '90 — the word-level n_0'.

- A. J. Menezes, P. C. van Oorschot, S. A. Vanstone, *Handbook of Applied Cryptography* — Algorithms 2.149, 4.44, 14.42, 14.82.

- N. Möller, *On Schönhage's algorithm and subquadratic integer GCD computation*, Math. Comp. 77 (2008).

- H. Cohen, *A Course in Computational Algebraic Number Theory* — Algorithms 1.7.1, 2.6.3; §3.3–3.4.
- R. Baillie and S. S. Wagstaff, *Lucas pseudoprimes*, Math. Comp. 35 (1980).
- M. O. Rabin, *Probabilistic algorithms in finite fields*, SIAM J. Comput. 9 (1980).
- D. Hankerson, A. Menezes, S. Vanstone, *Guide to Elliptic Curve Cryptography* — Algorithms 2.36, 2.48; §2.3.5.
- D. J. Bernstein, *How to find smooth parts of integers*, 2004.
- A. K. Lenstra, H. W. Lenstra, L. Lovász, *Factoring polynomials with rational coefficients*, Math. Ann. 261 (1982).
- J. von zur Gathen and J. Gerhard, *Modern Computer Algebra*, 3rd ed. — §5.10, §14 (Cantor–Zassenhaus).

The crate's own `CITATIONS.md` maps each routine to its primary source at finer grain.